Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will use import and def to create functions that split the program into steps, like asking for input, searching for flights, and printing results. I'll use while loops to keep asking the user for an airport or date until they enter a valid one. I'll use for loops to go through the list of flights and print the cheapest options. I'll also use if and else to handle different cases, like a one-way vs. round-trip search, or a search with no flights found.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the easiest part to be asking for the airports and dates and printing the results, since that only uses input(), print(), and f-strings that I've practiced in class, and the hardest part to be connecting to the flight API and reading its nested JSON response.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The tasks that will take the most time and effort are connecting to the flight API, reading its response, and testing different cases like round trips and invalid input, so I should implement the input and date validation first, then make one API call, then sort the prices to find the cheapest flight, and add extras like round trips and better error messages last.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

The biggest distraction while working on this project will probably be my phone and social media, so I'll put my phone on silent and work in short, focused blocks. For resources, I can get help from my professor and TAs, go to office hours when I get stuck, ask my friends, and use an AI chat assistant to explain things. I don't have any questions about the assignment requirements right now.

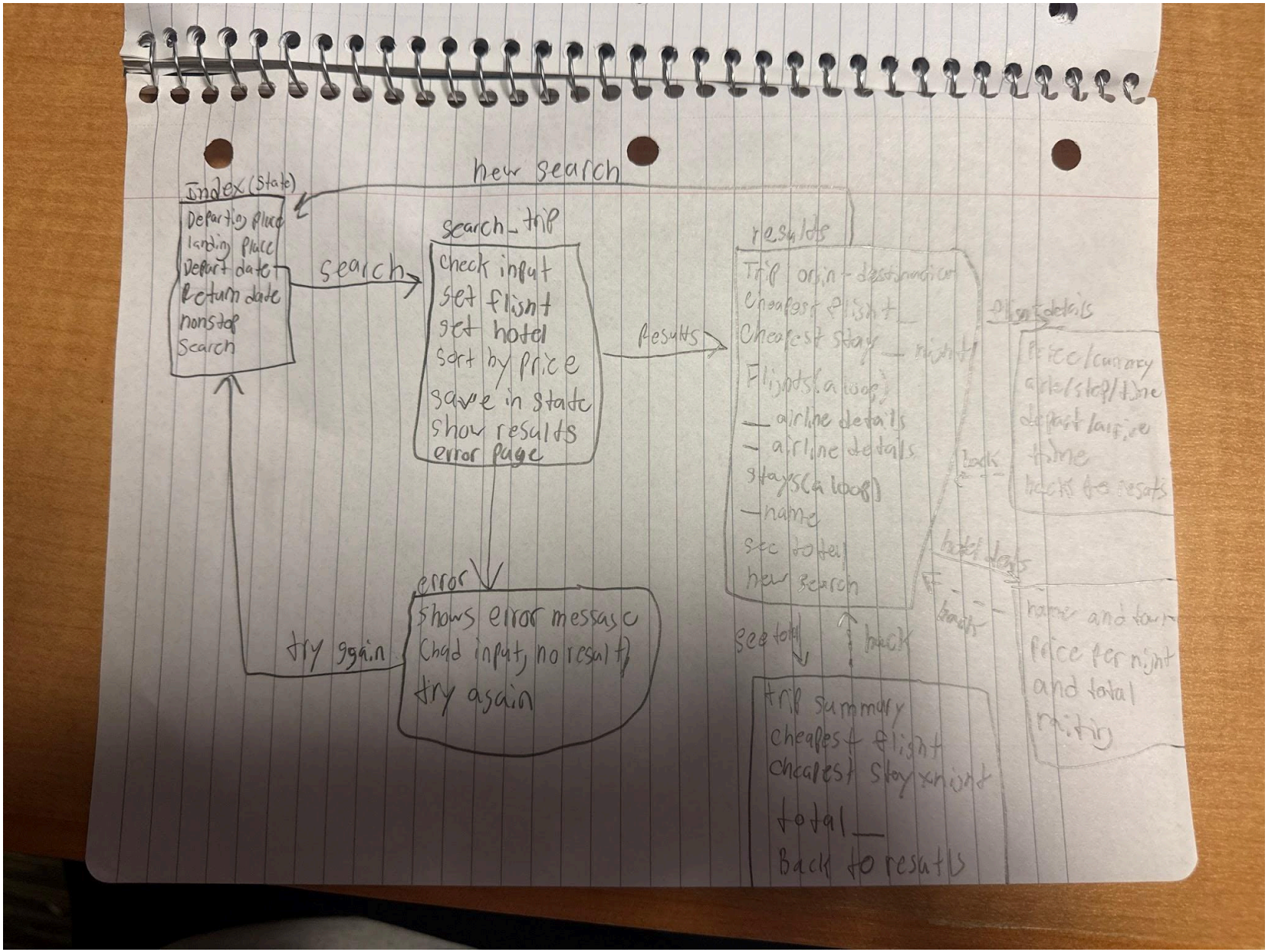
1) Site Diagram

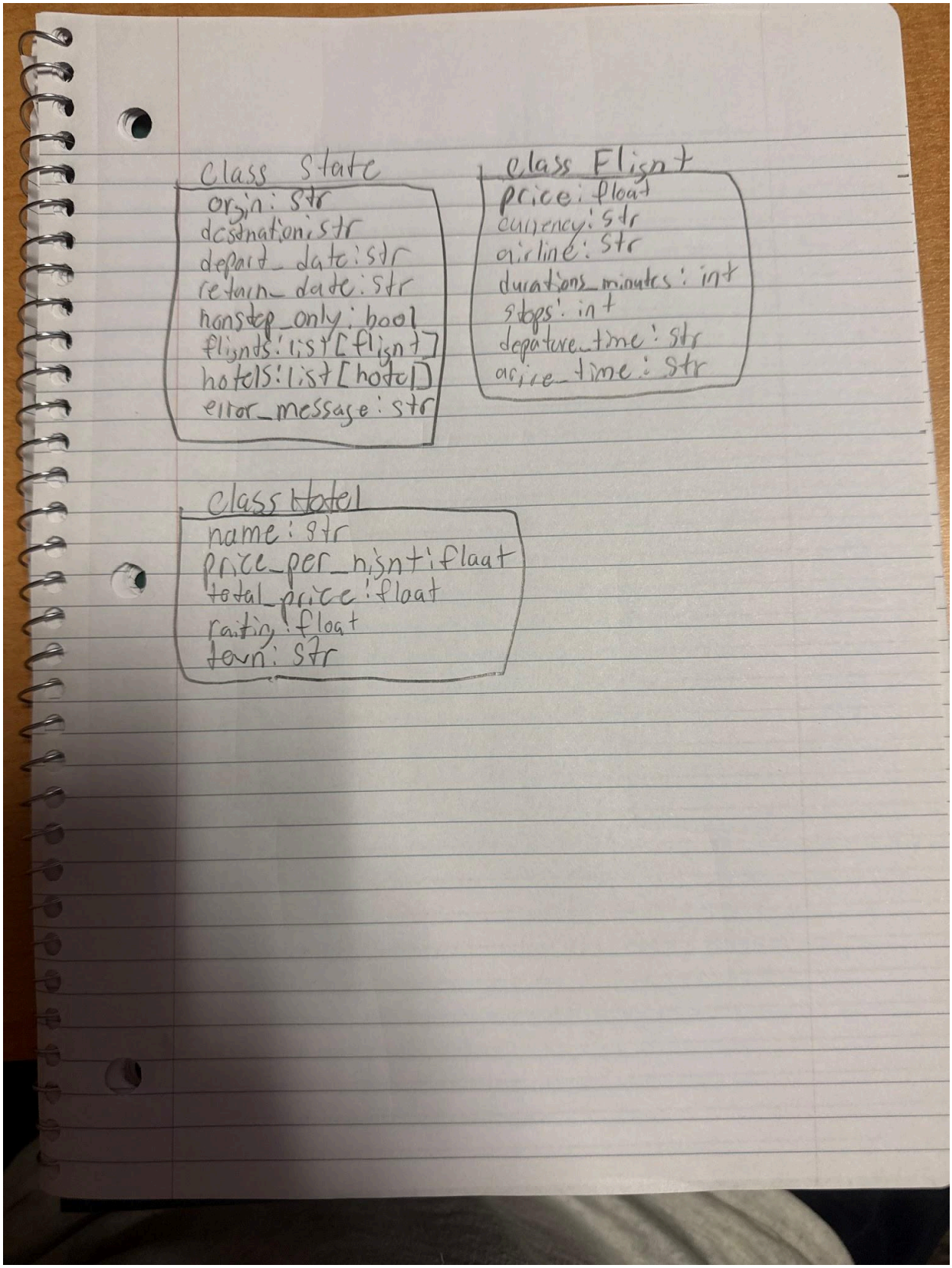
Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


new search
Index (State)
Departing Place
landing Place
Depart date
Return date
nonstop
Search
search
search_trip
check input
set flight
set hotel
sort by price
save in state
show results
error page
Results
results
Trip origin - destination
Cheapest flight
Cheapest stay — night
Flights(a loop)
— airline details
— airline details
stays(a loop)
— name
see total
new search
flight details
price/currency
airlines/stop/time
depart/arrive
time
back to results
back
hotel details
name and rating
price per night
and total
rating
error
shows error message
(bad input, no results)
try again
try again
see total
back
trip summary
cheapest flight
cheapest stay x night
total —
Back to results


class State
origin: str
destination: str
depart_date: str
return_date: str
nonstop_only: bool
flights: list[flight]
hotels: list[hotel]
error_message: str
class Flight
price: float
currency: str
airline: str
durations_minutes: int
stops: int
departure_time: str
arrive_time: str
class Hotel
name: str
price_per_night: float
total_price: float
rating: float
town: str

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**Cheapest Trip Finder lets a user enter where they are flying from and to, their dates, and a few preferences, then shows the cheapest flights and the cheapest places to stay at the destination. It helps budget travelers compare prices in one place and see the total cost of a trip**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	origin: str. The airport the user flies from. Saved when they press Search. destination: str. The airport the user flies to, and where to look for a place to stay. depart_date: str. The day they leave. return_date: str. The day they come back. Blank means one way. nonstop_only: bool. True if the box is checked, so flights with stops are removed. sort_by: str. "price" or "duration", so the flights are put in that order. flights: list[Flight]. The flights found. It is replaced every time they search. hotels: list[Hotel]. The places to stay found. It is replaced every time they search. error_message: str. The problem to show if something goes wrong, like a bad date or no results. The search_trip route changes all of these every time the user searches, and none of them stay the same.

Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Flight (one flight in State.flights) price: float. How much the flight costs. Used to find the cheapest one. currency: str. The money type, like "USD". airline: str. The airline name. duration_minutes: int. How long the trip takes. Used when sorting by duration. stops: int. How many stops it has. Used to hide flights with stops. depart_time: str. When the flight leaves. arrive_time: str. When the flight lands. Hotel (one place to stay in State.hotels) name: str. The name of the hotel. price_per_night: float. The cost for one night. total_price: float. The cost for the whole stay (nightly price times number of nights). Used to find the cheapest place. rating: float. The guest rating. neighborhood: str. The area where it is. How they are changed: search_trip makes a Flight or Hotel for each result it finds. They are not edited after that. Every new search replaces the whole list. Flight is the second dataclass and Hotel is a third.
Routes (minimum 7) The names of your route functions	index(state) search_trip(state, origin, destination, depart_date, return_date, nonstop_only, sort_by) results(state) flight_details(state, index) hotel_details(state, index) trip_summary(state) error_page(state)

Pages (minimum 5) A brief description of the pages in your site	index: home page with the search form. results: shows the cheapest flight and cheapest stay, the list of flights, and the list of stays, each with a Details button. flight_details: price, airline, stops, duration and times for one flight, with a Back button. hotel_details: name, neighborhood, nightly and total price and rating for one stay, with a Back button. trip_summary: the cheapest flight plus the cheapest stay and the trip total. error_page: shows the error message with a Try again button.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	TextBox: departing airport code, so the site knows where the flight starts. TextBox: destination airport code, used for the flight and for the area to find a place to stay. TextBox: departure date, used for the flight search and the start of the stay. TextBox: return date (optional). Leaving it blank means one way. CheckBox: nonstop flights only, to hide flights with stops. SelectBox: sort by price or duration, to choose how flights are ordered.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	1. In search_trip: if an airport code is not 3 letters, a date is invalid or in the past, or the return date is before the departure date, set error_message and show error_page instead of searching. 2. In search_trip: if return_date is empty, do a one-way search and show stays per night only. Otherwise do a round trip and calculate each stay's total from the number of nights. 3. In search_trip: if nonstop_only is True, remove flights with stops. If sort_by is "duration", sort by duration, otherwise by price.

	4. In search_trip: if no flights or stays come back, or the API fails, set error_message and show error_page. 5. In trip_summary: if there is no stay or no return date, show only the flight price and note that lodging is not included.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	In results, a for loop goes over state.flights (for flight in state.flights) and another goes over state.hotels. Each builds a row with the price, the airline or hotel name, and a Details button that holds its index. In trip_summary, a loop goes over the same lists and compares flight.price and hotel.total_price to find the cheapest of each. This is meaningful because the number of results changes with every search, so the page cannot use fixed rows, and the cheapest option has to be found from the data.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

it is for a compsci prodject that i am doing where i am making a code that you put in your dates and it tells you the cheapest flight to and from your desired locations can you do that?

is this possible

Yes! This is absolutely possible as a CS1 final project using Drafter and Python. I actually think it's a really good project idea because it has a clear purpose, takes user input, processes information, and produces useful results.

There is just one important thing we need to consider: how your program gets real flight prices.

1. How your flight finder would work

Imagine a user opens your website and sees something like this:

CheapFlight Finder

Flight search

Find the cheapest flight for your next trip!

Where are you flying from?

Newark (EWR)

Where do you want to go?

Orlando (MCO)

Departure date

November 20, 2026

Return date

November 25, 2026

Find Cheapest Flights

Example search results

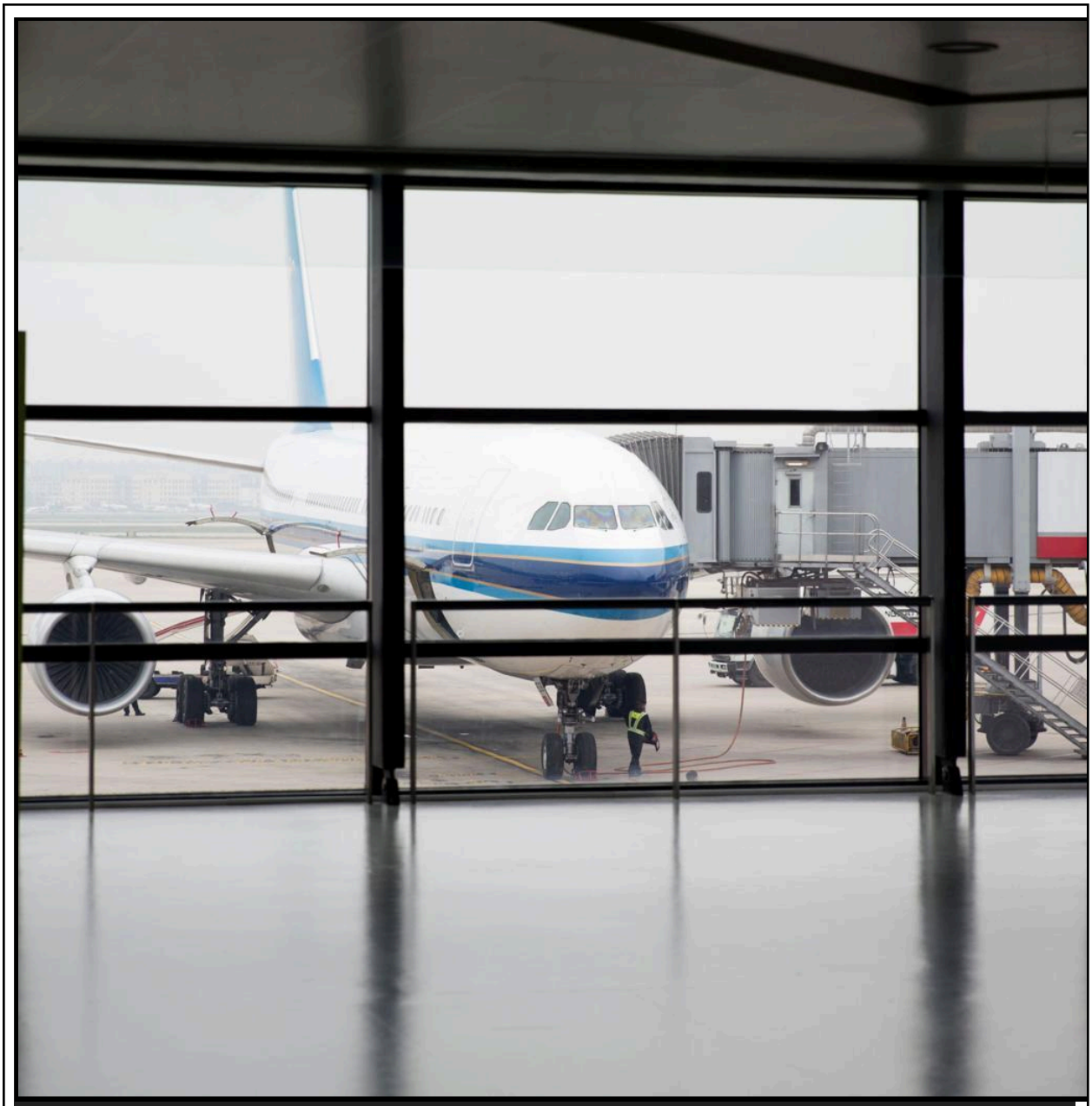


Airline A

Round trip · 1 stop

$145

Cheapest



Airline B

Round trip · Direct

$189

Illustrative example only, not live flight prices.

2. Is it actually possible with Drafter?

Yes, but there are two approaches.

Option A: Use sample flight data (easiest and most reliable)

You create a collection of example flights in Python.

Your program:

1. Takes the departure location.
2. Takes the destination.
3. Takes the departure and return dates.
4. Looks through a collection of flights.
5. Filters out flights that don't match.
6. Finds the cheapest matching flight.
7. Displays the results.

This is very achievable for a CS1 student.

Option B: Use real flight prices (more complicated)

You could potentially connect to a flight-search API that provides actual flight information.

However, there are challenges:

- Some APIs require paid subscriptions.
- Some require API keys.
- Some have usage limits.
- Some don't support direct browser requests.
- Drafter runs Python in the browser and doesn't have its own backend.

I would not make live flight data a requirement for your project.

3. What I recommend building

For a three-week introductory programming project, I recommend starting with Option A and making the application look and feel like a real flight-search website.

Here are the features I'd include:

Feature	Difficulty
Enter departure location	Easy
Enter destination	Easy
Select departure date	Easy
Select return date	Easy
Store example flight information	Easy
Filter matching flights	Medium
Sort flights by price	Easy
Display cheapest flight	Easy
Display multiple flight options	Easy
Connect to live airline prices	Hard

4. How the Python processing would work

For example, imagine your program has these flights:

```
flights = [
  {"from": "Newark", "to": "Orlando", "price": 145},
  {"from": "Newark", "to": "Orlando", "price": 189},
  {"from": "Newark", "to": "Miami", "price": 120},
  {"from": "Newark", "to": "Orlando", "price": 165}
]
```

The user enters Newark and Orlando.

Your program checks each flight, keeps the matching flights, and sorts them by price.

The cheapest flight would be $145.

You could eventually add departure dates, return dates, airlines, and flight durations.

5. Let's make sure your project is scoped correctly

Your uploaded assignment asks you to explain your problem, prior solutions, IPO model, data abstraction, challenges, and project goals.

I think your idea fits those requirements very well.

One question before we start planning the project:

Do you want your program to find flights using:

A. Example flight data

The program uses a collection of flights with made-up prices. Much easier to build and test.

B. Real flight prices

The program attempts to retrieve actual flight prices from the internet. More challenging.

C. Both

Start with example data, then attempt to add real flight prices as an advanced feature.

My recommendation is C. Build something that definitely works first, then see if you can add live flight prices. That way, you have a working project even if connecting to flight APIs becomes too difficult.

I want to attempt real flight prices. Help me assess the API and Drafter limitations.

I think attempting real flight prices is a great way to make your project stand out. We just need to be careful about choosing an API that works with Drafter.

The good news is that your idea is technically possible, but there is a major limitation: Drafter runs Python in the user's browser using Pyodide. It does not have its own

Python server or database.

That changes which flight APIs you can realistically use.

1. How your application would work

Your planned application would have this structure:

User enters flight information

Departure · Destination · Departure date · Return date

Drafter / Python

Collects inputs and sends a request

Flight-search API

Returns available flights and prices

Python compares prices

Sorts flights from cheapest to most expensive

Displays cheapest flights

Airline · Price · Flight duration · Booking link

The most important part is finding an API that allows your browser-based application to request flight information.

2. Which flight API should you use?

I researched two realistic options.

Option 1: Amadeus for Developers

My recommendation



Amadeus offers a Flight Offers Search API that can return:

- Flight prices
- Airline names
- Departure and arrival times
- Flight durations
- Available flight options
- One-way and round-trip searches

Advantages

- Has a free testing environment with a monthly request allowance.
- Designed for developers who are new to travel technology.
- Provides actual flight-search functionality.

Disadvantages

- The test environment uses limited datasets, not complete live flight inventory.
- Its self-service results exclude American Airlines, Delta, British Airways, and low-cost carriers.
- Its API authentication requires credentials that should not be exposed in your browser code.

Sources:

Amadeus for Developers

+2

Amadeus developer website

Option 2: Duffel



Duffel provides an API that searches airline offers and returns flight prices and itineraries.

Advantages

- Designed specifically for flight searches.
- Provides detailed flight information.
- Supports return trips and multiple flight segments.

Disadvantages

- Requires an API access token.
- Explicitly blocks direct browser requests.
- Requires a separate backend to protect your access token.

My verdict: Not suitable for a Drafter-only project.

Sources:

Duffel Documentation

+1

Duffel website

3. The biggest problem: Drafter has no backend

This is the most important technical issue for your project.

Drafter runs your Python code inside the visitor's browser. That means if you put an

API password or secret key in your Python code, someone could potentially inspect the application and steal it.

Drafter Documentation

+2

Here is what that means:

Feature	Drafter support
User enters travel dates	Yes
User enters airports	Yes
Python sends internet requests	Yes, if CORS permits
Python processes API results	Yes
Sort flights by price	Yes
Securely hide private API credentials	No
Run your own backend server	No

Drafter's documentation confirms that Python's requests library can access internet APIs, provided the API allows browser-based requests through CORS.

Drafter Documentation

+1

This means you can technically connect to some APIs, but you cannot safely use an API that requires a private secret directly from your application.

4. What are our possible solutions?

I see three possible approaches.

Best if backend is not allowed

Approach A: Public flight data API

Find an API that provides flight information without requiring a private secret and permits browser requests.

- Drafter sends the request.
- API returns flight data.
- Python sorts the results.

The challenge is finding a legitimate provider with current flight prices and suitable access.

Requires instructor approval

Approach B: A tiny API proxy

A small separate service holds the API credentials and communicates with Amadeus.

- Drafter sends the search details.
- The proxy securely contacts Amadeus.
- The proxy sends flight results back to Drafter.

This is technically the better architecture for real flight prices, but it introduces a backend, which may violate your assignment constraints.

Good fallback

Approach C: Flight search links

Your application collects the dates and locations and creates a link to a flight-search website such as Google Flights.

This does not retrieve prices directly into Python, but allows users to continue searching for real flights.

5. My recommended development plan

I would build this in stages so you don't get stuck trying to solve everything at once.

Stage	What you build
1	Create the flight-search interface in Drafter
2	Let users enter airports and travel dates
3	Investigate and test Amadeus API access
4	Connect Drafter to the API if access is permitted
5	Extract flight prices from the returned information
6	Sort results from cheapest to most expensive
7	Display the cheapest three flights

One important detail: your program should describe its result as the cheapest flight among the offers returned by the API, rather than claiming it found the cheapest flight across every airline.

Also, I would leave actual flight booking out of your project. Finding and comparing flights is already a substantial application.

6. Two things I need to know before we proceed

These will determine which technical approach we should take.

Question 1: Are you required to use only Drafter?

Yes. We cannot use any backend or outside hosting services.

We might be allowed to use a small separate service if necessary.

I'm not sure. I can ask my professor.

Question 2: What kind of flights should your application find?

Round-trip flights (there and back)

One-way flights

Both one-way and round-trip flights

My goal is to help you build something that actually works, uses real flight information if technically possible, and meets your CS1 project requirements without becoming too complicated.

what will be the hardest part

The hardest part of your project will most likely be connecting your Drafter application to the flight API and getting real flight prices back.

Your actual Python programming (sorting flights, comparing prices, finding the cheapest option) is relatively straightforward. The difficult part is getting the flight

information in the first place.

Here is how I would rank the challenges:

The 5 hardest parts

1

Connecting to the flight API

Hardest

- Finding an API that provides real flight prices.
- Learning how to send requests.
- Making sure the API works with Drafter and Pyodide.
- Handling API authentication securely.

This is the biggest technical risk.

2

Understanding the API response

- The API returns information in JSON format.
- You need to extract prices, airlines, dates, and flight durations.
- Some flights might have missing information.

You'll need to learn how to work with dictionaries and lists.

3

Comparing and sorting flight prices

- Compare different flight prices.
- Sort them from cheapest to most expensive.
- Display the best options.

This should be manageable with basic Python.

4

Handling dates and user input

- Make sure dates are valid.
- Prevent return dates before departure dates.
- Make sure airport codes are valid.

5

Designing the website

- Create the search page.
- Display flight options attractively.
- Make the website easy to navigate.

Drafter should make this manageable.

What I would write in your project assignment

Your assignment asks you to identify the technically difficult parts of your project.

Here is a draft you could use:

The most difficult part of my project will likely be connecting my Drafter application to an external flight-search API to retrieve real-time flight prices. I will need to learn how APIs work, how to send requests, and how to interpret the information returned in JSON format. Another challenge will be making sure the API works with Drafter, since Drafter runs Python in the browser and does not have its own backend. I will also need to securely handle my API key. Once I successfully retrieve flight information, I think comparing prices and sorting flights from cheapest to most expensive will be more manageable.

My biggest piece of advice

Build your project in this order:

1. Get the Drafter website working.
2. Get the API connection working.
3. Retrieve flight prices.
4. Sort the flights by price.
5. Make the results look nice.

Don't spend two weeks designing the website before testing the API. If the API connection doesn't work, it could affect your entire project.

I would actually make testing the API your very first milestone.

For building a flight (and hotel) finder, here's a travel data API option.

what should my code look like if i also want to have a cheapest hotel near the saod destination aswell

Absolutely — you want each section formatted as a Requirement | Implementation two-column table, with answers you can copy and paste directly into your assignment.

Cheapest Trip Finder: Planning Document

Purpose / Functionality

Requirement	***Implementation***

Purpose of the website	*Cheapest Trip Finder allows users to enter their departure airport, destination, departure date, and return date. The website searches for the cheapest available flights and hotels near the destination. It then compares the prices and calculates an estimated total trip cost to help users plan affordable vacations.*

State Dataclass Attributes

Attribute	*Type*	*Description and how it is modified*
origin	*str*	*Stores the airport code the user is departing from. Updated when the user submits a search.*
destination	*str*	*Stores the destination airport code. Also used to search for nearby hotels.*
depart_date	*str*	*Stores the departure date entered by the user. Used to search for flights and calculate the hotel stay.*
return_date	*str*	*Stores the return date. If left blank, the program searches for a one-way flight.*
nonstop_only	*bool*	*Stores whether the user wants nonstop flights only. Used to filter flight results.*
sort_by	*str*	*Stores whether flights should be sorted by price or duration.*
flights	*list[Flight]*	*Stores the flight results returned by the flight API. Updated whenever a new search is performed.*

hotels	*list[Hotel]*	*Stores hotel results returned by the hotel API. Updated whenever a new search is performed.*
error_message	*str*	*Stores an error message if the search fails, the inputs are invalid, or no results are found.*

Second Dataclass

Attribute	*Type*	*Description and how it is modified*
Flight.price	*float*	*Stores the price of a flight. Used to compare and sort flights.*
Flight.currency	*str*	*Stores the currency used for the flight price.*
Flight.airline	*str*	*Stores the airline name displayed to the user.*
Flight.duration_minutes	*int*	*Stores the total flight duration. Used when sorting by travel time.*
Flight.stops	*int*	*Stores the number of stops. Used to filter nonstop flights.*
Flight.depart_time	*str*	*Stores the departure time of the flight.*
Flight.arrive_time	*str*	*Stores the arrival time of the flight.*

Flight objects are created from the information returned by the flight API and stored in

state.flights. The list is replaced whenever the user performs another search.

Hotel dataclass (additional dataclass)

Attribute	Type	Description and how it is modified
Hotel.name	str	Stores the hotel's name.
Hotel.price_per_night	float	Stores the hotel's nightly price.
Hotel.total_price	float	Calculates the total hotel cost based on the number of nights.
Hotel.rating	float	Stores the hotel's customer rating.
Hotel.neighborhood	str	Stores the neighborhood where the hotel is located.

Hotel objects are created using information returned by the hotel API and stored in state.hotels. The hotel prices are compared to identify the cheapest accommodation.

Routes

Route	Description
index(state)	Displays the homepage and flight/hotel search form.
search_trip(state, ...)	Processes the user's inputs and searches for flights and hotels.

results(state)	*Displays the available flights and hotels sorted by price.*
flight_details(state, index)	*Displays detailed information about a selected flight.*
hotel_details(state, index)	*Displays detailed information about a selected hotel.*
trip_summary(state)	*Displays the cheapest flight, cheapest hotel, and estimated total trip cost.*
error_page(state)	*Displays error messages when something goes wrong during a search.*

Pages

Page	***Description***
index	*The homepage where users enter their departure airport, destination, travel dates, and preferences.*
results	*Displays the cheapest flights and hotels found, along with other available options.*
flight_details	*Shows additional information about a selected flight, including airline, price, duration, and stops.*
hotel_details	*Shows information about a selected hotel, including its name, rating, location, and price.*

trip_summary	*Combines the cheapest flight and hotel prices to show the estimated total cost of the trip.*
error_page	*Displays an error message if the search fails or no flights or hotels are found.*

User Input

Component	*Purpose*
TextBox	*Allows users to enter their departure airport code.*
TextBox	*Allows users to enter their destination airport code.*
TextBox	*Allows users to enter their departure date.*
TextBox	*Allows users to enter their return date. Leaving it blank allows a one-way search.*
CheckBox	*Allows users to select whether they only want nonstop flights.*
SelectBox	*Allows users to choose whether flight results should be sorted by price or duration.*

If Statements

If Statement	*Description*

Input validation	*Checks whether airport codes and dates are valid. If the return date is before the departure date or the information is invalid, the program displays an error message.*
One-way vs. round-trip	*Checks whether the return date is blank. If it is blank, the program searches for a one-way flight. Otherwise, it searches for a round-trip flight.*
Nonstop filter	*Checks whether the user selected the nonstop-only option. If true, flights with stops are removed from the results.*
Sorting preference	*Checks whether the user selected price or duration. The program sorts the flights based on the selected preference.*
Search results	*Checks whether the APIs returned any flights or hotels. If no results are found or an API request fails, the program displays an error page.*
Trip summary	*Checks whether hotel results are available and whether the user entered a return date. If not, the program explains that lodging is not included in the total.*

Loop

Requirement	***Implementation***
Flight loop	*A for loop iterates through state.flights and displays each flight's airline, price, duration, and number of stops. This allows the website to display different numbers of flights depending on the search results.*

Hotel loop	*A for loop iterates through state.hotels and displays each hotel's name, nightly price, rating, and location.*
Finding cheapest options	*The program loops through the flight and hotel lists to compare prices and identify the cheapest flight and cheapest hotel. These results are used to calculate the estimated total trip cost.*

Important technical note for your project: This is a proposed design, not confirmation that both APIs are available. You will need a working flight API and hotel API, and a secure way to access them. Also, your original assignment requires at least one list of dataclass instances in State; flights satisfies that requirement. The design includes three dataclasses total: State, Flight, and Hotel.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.